Solutions OS Revamp

From a collection of fragmented tools to a unified AI development platform — one repo, one command, every capability.

Proposal for Approval · Author: Ram · 4 interconnected changes · 6-week rollout

ACT 1 · CONTEXT

WHERE WE ARE TODAY

What is Solutions OS?

A shared repository that gives AI coding agents deep knowledge about our Appian applications — enabling AI-assisted development, design docs, code review, test data generation, and more.

Knowledge Layer

Product context, feature specs, personas, architecture decisions — organized per product.

AI Tools (MCP Servers)

Cloud Plane, Live Plane, Data Generator — give AI agents real access to our Appian environments.

Powers (Capabilities)

Pre-built AI workflows — developer, product owner, UX designer, SQL forge, and more.

In one line: **Solutions OS makes AI agents experts on our Appian products** — so a developer, PO, or QE can get useful, context-aware help in seconds instead of reading code for hours.

ACT 1 · CONTEXT

IT ALREADY WORKS

20 projects built on this platform

During SWAT-a-Palooza, teams built **20 working projects** on Solutions OS — spanning the full software lifecycle. The value is proven. The question is how to scale it to everyone.

5	Dev tooling projects
5	Docs & reporting
4	Design & UX handoff
2	Test & QE
2	Accessibility
2	Data generation

The takeaway: Solutions OS has proven its value across every discipline. But scaling it from a hackathon to *all Solutions teams* has exposed structural problems — which is exactly what this revamp addresses.

ACT 2 · THE PROBLEMS

WHY NOW

Four structural problems slow adoption

As Solutions OS moves to production across all teams, four problems have surfaced. They share a root cause: **the platform grew organically as separate tools, never designed as one system.**

1. **Tool fragmentation**
   Two tools (Atlas & Jarvis) solve the same problem with different backends. 6+ capabilities are duplicated.

2. **Three overlapping tools for the live environment**
   Jarvis, buildwithclaude, and lcp-api all talk to live Appian. Users pick one and miss the others.

3. **Painful onboarding — 30 to 60 minutes**
   Manual power install, MCP config, Docker auth, env vars — repeated for every single power.

4. **Unmaintainable at scale**
   Every power bundles its own MCP config. Same server duplicated across powers. No central control.

Net result: most users install one power, never discover the rest. Capabilities stay siloed, and the support burden grows with every new power. We'll look at each problem, then the solution.

ACT 2 · THE PROBLEMS

PROBLEM 1 — FRAGMENTATION

Atlas vs Jarvis: two tools, one job

Both answer the same question — *"give AI agents structured knowledge about Appian applications"* — but they were built independently with different storage strategies.

DIMENSION	ATLAS (CLOUD KB)	JARVIS (LIVE ENV)
Storage	GitLab — versioned JSON	Live Appian environment
Multi-release history	✓ Full history & diffs	✕ Current state only
Offline analysis	✓ No connection needed	✕ Requires live env
Freshness	Point-in-time snapshots	✓ Always latest
Overlapping tools	6+ duplicates: overview, search, code, dependencies, dead-code, impact	

The key insight: these aren't competitors — they're **complementary data planes**. Cloud excels at history & offline analysis; Live excels at real-time state. The problem is they were built separately, so a power built on one can't use the other.

ACT 2 · THE PROBLEMS

PROBLEM 2 — LIVE ENV OVERLAP

Three tools all talk to live Appian

Three independent projects give AI agents access to live environments. They overlap heavily but sit on different foundations — so which do we standardize on?

Jarvis

Custom Appian app deployed per environment. 42 tools.

- Semantic search, clusters, patterns
- Dead code & staleness tracking
- Package create/deploy

Needs: app deployed on target env

buildwithclaude

MCP server over OOTB LCP APIs. 150+ tools.

- Full CRUD on design objects
- SAIL eval, test expressions
- Record data operations

Needs: LCP plugin on site

lcp-api (a!migo)

Python library over OOTB LCP APIs. 237 operations.

- 130 plugin + 107 beta operations
- Multi-profile (dev/staging/prod)
- Google Sheets → data model

Needs: LCP plugin + toggles

The problem: three teams built three ways to talk to live Appian. The overlap is large, but each has unique strengths — so we need to decide *which tool owns which responsibility* rather than maintaining all three in parallel.

ACT 2 · THE PROBLEMS

PROBLEM 3 — ONBOARDING

30–60 minutes before the first useful answer

Here's what a new team member goes through today just to start using one capability:

Clone repo → Navigate folder maze → Find the right power → Install power → Configure MCP server → Set env vars →

Docker login → Docker pull → Verify connection → Repeat for every other power

⏱️ 30–60 minutes

To first useful AI interaction — if nothing goes wrong. SSL errors, Docker auth failures, and wrong env vars are common.

📦 3+ MCP configs per user

Each power adds its own server entries. Three powers using the same server = three identical Docker containers, configured separately.

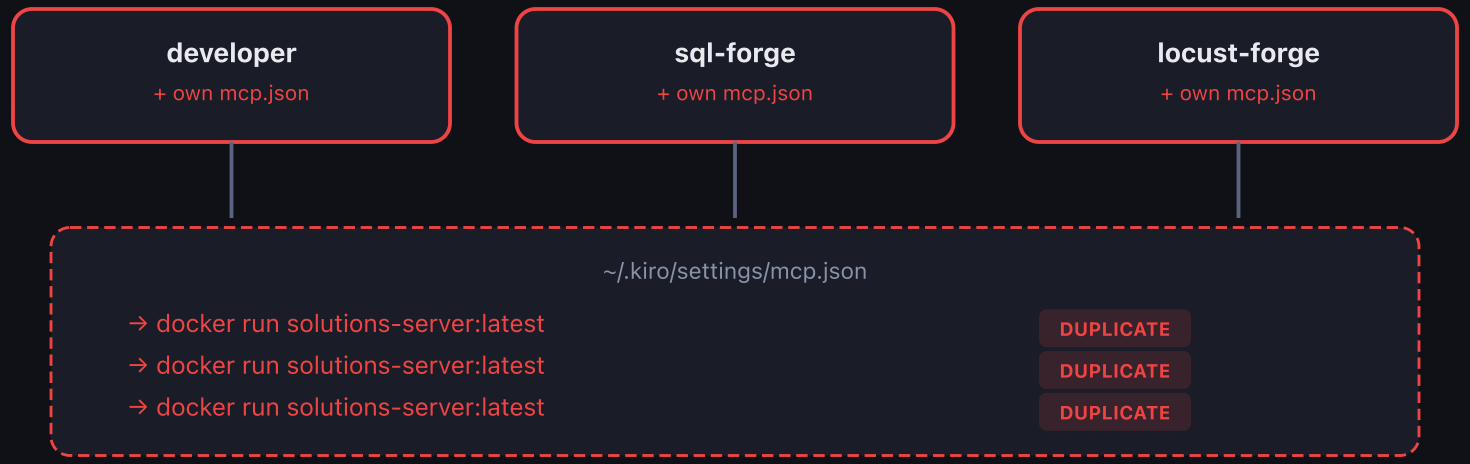
> *"I installed the developer power but didn't know sql-forge and ux-designer existed. Each one was another 15 minutes of setup."*

ACT 2 · THE PROBLEMS

PROBLEM 4 — MAINTAINABILITY

Every power bundles its own MCP config

This is the root cause of the onboarding pain — and it gets worse with every power added.



Same image duplicated 3×. Credentials scattered. Updates break.

Power authors must understand Docker, the MCP protocol, and credential management.

Adding a power shouldn't require Docker and MCP expertise. **The fix is to treat MCP servers as shared infrastructure, not power-scoped resources** — which is exactly what the solution does.

ACT 3 · THE SOLUTION

THE PLAN

Four interconnected changes

Each change reinforces the others. Together they turn Solutions OS from a fragile tool collection into a production-grade platform. We'll walk through each one.

1 **Modular MCP — Read · Write · Deploy**
Split tools by verb. One Intelligence Server reads; a!migo writes; Deployment MCP ships. Reuse what's already maintained.

2 **Single Orchestrator Agent**
One entry point that routes to specialist sub-agents. No manual power selection.

3 **Global Config Bootstrap**
One setup script installs everything. MCP = infrastructure. Powers = lightweight knowledge.

4 **Repository Restructure (T.I.M.E.)**
Lifecycle folders, enforced conventions, single source of truth.

METRIC	TODAY	AFTER REVAMP
Time to first AI interaction	30–60 min	< 5 min
MCP configs to manage	3+ per user	1 (unified)
Powers that use both backends	0	All of them
Adding a new power	Folder + mcp.json + Docker + docs	Folder + POWER.md. Done.

ACT 3 · THE SOLUTION

THE ORGANIZING IDEA

Separate by verb: Read · Write · Deploy

The single decision that resolves both the fragmentation and the "three Live tools" problem: **stop merging everything into one tool, and split responsibilities by what the agent is doing.**

READ

Intelligence Server

Understand the application. Cloud Plane (history) + Live Plane (real-time). Read-only — never modifies anything.

WRITE

a!migo (lcp-api)

Create & modify objects via Appian's own OOTB APIs. 237 operations, already maintained.

DEPLOY

Deployment MCP

Package, deploy, and promote between environments. Open-source, dedicated.

Why split instead of building one monolith? Each module gets a single, clear responsibility — and we *reuse existing maintained projects* for writes and deploys instead of reinventing them. The orchestrator simply routes by the verb in the request.

ACT 3 · THE SOLUTION

UNIFIED IDENTITY

One Unified MCP: Solutions Intelligence

Before the details — a quick naming reset. "Atlas" and "Jarvis" served well as prototypes. Going to production, components are named by *what they do*, not by codename.

OLD NAME	NEW NAME	WHAT IT IS
~~Atlas MCP Server~~	**Solutions Intelligence Server**	The unified read-only MCP server
~~Atlas (Cloud KB)~~	**Cloud Plane**	Versioned, offline-capable layer (GitLab KB)
~~Jarvis (Live env)~~	**Live Plane**	Real-time, per-environment layer (Appian APIs)
~~Atlas Parser~~	**Solutions Parser**	KB generation engine (.zip → JSON)
~~JAI App~~	**Solutions KB App**	Appian app for live KB & staleness tracking
~~atlas-developer, atlas-sql-forge...~~	**developer, sql-forge...**	Powers named by function, not backend

Principle: components are named by what they do. This makes the platform self-documenting and approachable to new team members. The rest of this deck uses the new names.

ACT 3 · THE SOLUTION

CHANGE 1 — READ · CLOUD PLANE

Intelligence Server: the Cloud Plane

The Cloud Plane keeps the existing Atlas architecture — pre-parsed application data stored as versioned JSON in GitLab. Fast, offline-capable, multi-release.

How it works

1. Appian .zip packages exported from environments
2. **Solutions Parser** extracts code, dependencies, schema, patterns
3. Structured JSON committed to `solutions-kb`
4. Cloud Plane reads from GitLab (cached, fast)

What it provides

- Multi-release history & changelogs
- Cross-release comparison & diffs
- Pre-computed dependency graphs
- Dead code / orphan analysis
- Schema snapshots & relationships
- Hub objects & dependency paths

Strengths: works offline · shared across the team (one KB serves everyone) · full version history · pre-computed analysis with no runtime cost. Already operational with 6+ apps indexed.

ACT 3 · THE SOLUTION

CHANGE 1 — READ · LIVE PLANE

Intelligence Server: the Live Plane

The Live Plane is Jarvis, **streamlined to read-only**. It serves application knowledge from the live environment — all CRUD and deployment tools are removed and handed to dedicated servers.

Stays in the Live Plane (read)

- Real-time object code & content
- Live dependency chains & impact analysis
- App tree & object search
- Clusters, patterns, architecture
- Semantic search on live data
- Staleness tracking

Removed (moves elsewhere)

~~Object creation~~

~~Object modification~~

~~Package creation & deployment~~

~~SAIL expression evaluation~~

~~SQL queries~~

→ to dedicated write/deploy servers (next slide)

Result: the Intelligence Server (Cloud + Live) answers exactly one question — *"help me understand this application"* — and never modifies anything. A request auto-routes to the right plane: Cloud for history, Live for current state.

ACT 3 · THE SOLUTION

CHANGE 1 — WRITE & DEPLOY

The removed capabilities are already solved

We don't reinvent what's already built and maintained. The write and deploy concerns go to dedicated, independently-maintained servers.

✏️ a!migo (lcp-api) — WRITE

- 237 CRUD operations via OOTB LCP APIs
- Create/update record types, interfaces, rules
- Modify process models, constants, groups
- SAIL evaluation & SQL queries
- Data model from Google Sheets
- Multi-profile environment switching

Maintained by Saurabh · uses Appian's own OOTB APIs

🚀 Deployment MCP — DEPLOY

- Create deployment packages
- Inspect package contents
- Deploy to target environment
- Check deployment results
- Promote between environments

Open source · github.com/kelseymross/appian-deployment-mcp

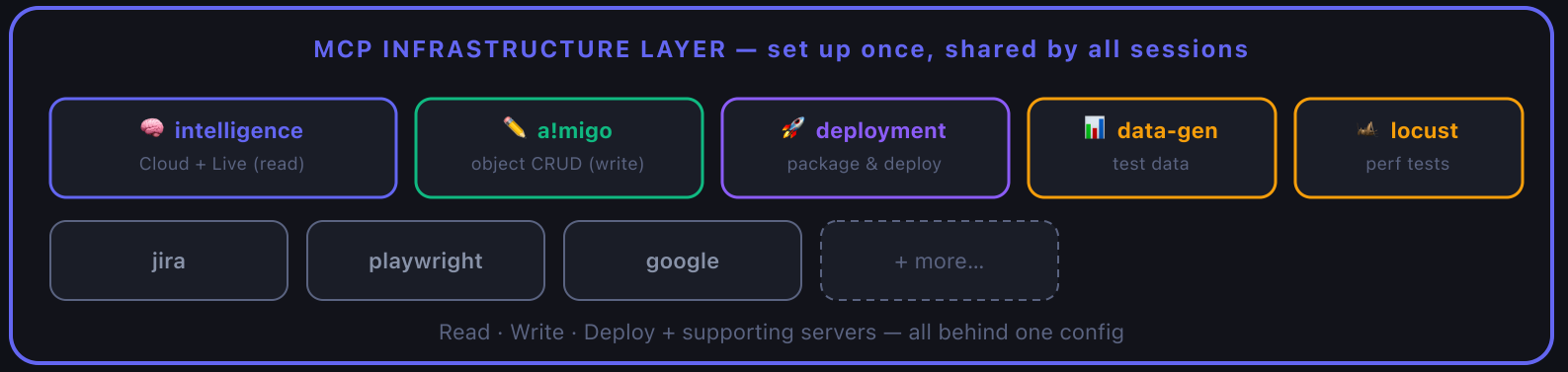
Principle — don't reinvent: the Intelligence Server reads, a!migo writes, Deployment MCP ships. Each module stays focused, maintenance is distributed across owners, and the orchestrator routes by the verb.

ACT 3 · THE SOLUTION

CHANGE 1 — THE FULL PICTURE

The complete MCP ecosystem

All servers configured once as shared infrastructure. Every agent can call any of them.



Maintenance is distributed: Intelligence (Ram + Soma), a!migo (Saurabh), Deployment (open source), Data Gen & Locust (Ram). Users experience it as **one unified set of tools**.

ACT 3 · THE SOLUTION

CHANGE 2 — ORCHESTRATOR

A single orchestrator agent

Users never choose which power to activate. The orchestrator understands the request and either handles it directly or delegates to the right specialist.

How it routes

"*How does scoring work in GSS?*"

→ handles directly (read tools)

"*Generate a design doc for GAMS-7126*"

→ delegates to developer

"*Create test data for evaluations*"

→ delegates to sql-forge

"*Fix accessibility on this interface*"

→ delegates to a11y-fixer

Available specialists

ENG **developer** — code, impact, design docs

PRODUCT **product-owner** — specs, release summaries

UX **ux-designer** — prototypes, Aurora compliance

DATA **sql-forge** — test data, ERDs, SAIL→SQL

QE **qe-agent** — test execution, verification

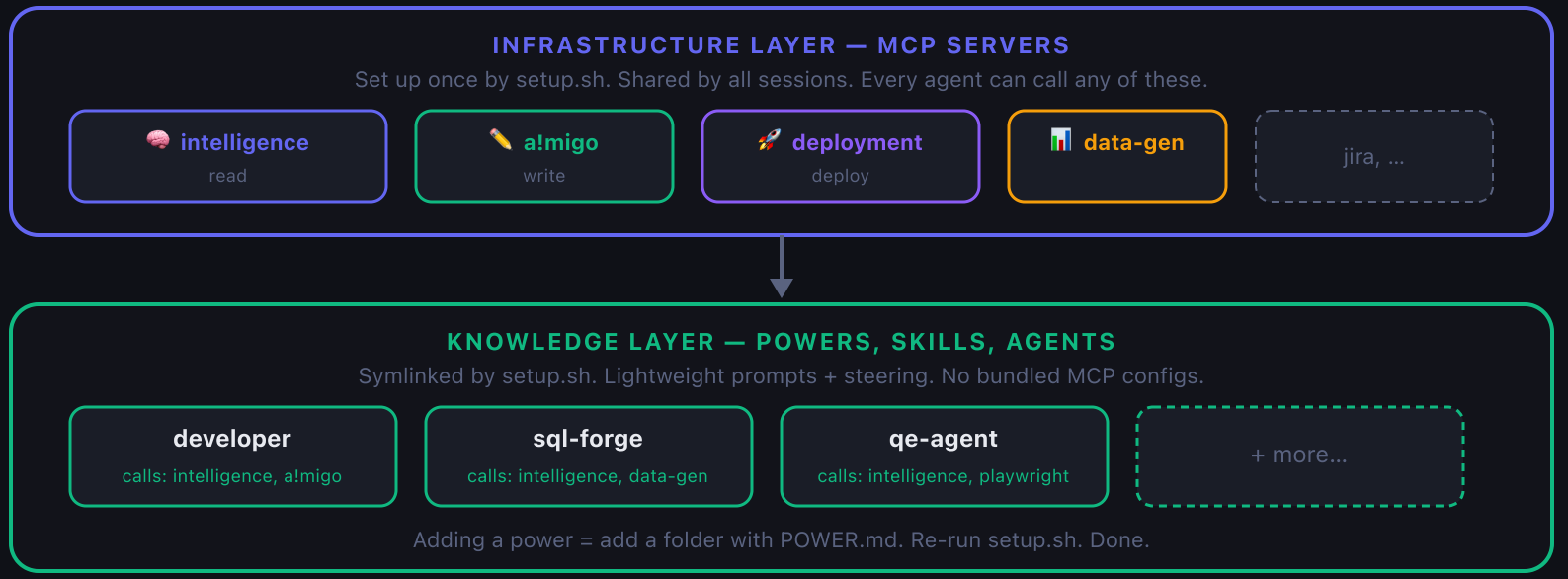
A11Y **a11y-fixer** — automated a11y fixes

ACT 3 · THE SOLUTION

CHANGE 3 — THE KEY ARCHITECTURAL CHANGE

Global Configuration Bootstrap

> *MCP servers are infrastructure, not power-scoped resources. Powers are lightweight knowledge that references tools already available.*



ACT 3 · THE SOLUTION

CHANGE 3 — IN PRACTICE

One command does everything

git clone → ./setup.sh → Start coding with AI

What `./setup.sh` does

1. Checks prerequisites (Docker, Kiro CLI)
2. Creates `.env`, validates credentials
3. Pulls Docker images for all MCP servers
4. Writes unified MCP config globally
5. Symlinks powers, skills, agents, steering
6. Registers everything & verifies it works

Maintenance commands

```
./setup.sh # Full install
./setup.sh --update # After git pull
./setup.sh --verify # Health check
./setup.sh --uninstall # Clean removal
./setup.sh --profile eng# Eng only
```

Why it works

- ✓ **Idempotent** — safe to re-run
- ✓ **No drift** — symlinks track git
- ✓ **No orphans** — clean uninstall
- ✓ **Credential isolation** — secrets in .env only

ACT 3 · THE SOLUTION

CHANGE 3 — IMPACT

What changes for power authors

Powers become dramatically simpler to create and maintain — this is the payoff for the Problem 4 maintainability issue.

Before: heavy powers

```
my-power/
├── POWER.md
├── steering/
├── mcp.json ← Docker config
│   ├── cloud server ← credentials
│   ├── live server ← credentials
│   └── data-gen server← credentials
```

Must understand: Docker, MCP protocol, registry auth, credentials, image tags

After: lightweight powers

```
my-power/
├── POWER.md
└── steering/
```

That's it.

Steering just says:
"Use get_app_overview to understand apps"
"Use search_objects to find code"

Tools already run. No config needed.
Author only needs to know: prompting.

Adding a new power: create a folder with POWER.md + steering, add one line to the manifest, re-run `./setup.sh`. No Docker knowledge. No MCP expertise. No credential handling.

ACT 3 · THE SOLUTION

CHANGE 4 — REPOSITORY STRUCTURE

Repository Restructure + T.I.M.E.

Products organized by lifecycle stage. The folder structure itself becomes a workflow signal — agents know what to do based on where a file lives.

T.I.M.E. lifecycle per product

```
products/<product>/
├── 00-context/ ← ground truth
├── 01-discovery/ ← raw ideas
├── 02-refinement/ ← specs + prototypes
├── 03-planning/ ← committed work
├── 04-delivery/ ← active dev
└── 05-shipped/ ← released
```

AI acts on transitions

→ discovery	scan for duplicates, flag dependencies
→ refinement	expand into spec, draft prototype
→ planning	break into tickets, estimate
→ delivery	generate design doc + tasks
→ shipped	write release notes, archive

Users just move files between folders. AI responds automatically. No commands to memorize.

ACT 4 · WHY THIS DESIGN WINS

ADVANTAGE 1 — EXTENSIBILITY

A living, extensible ecosystem

Solutions OS is a central repository for **every kind of AI capability** — powers, skills, agents, steering.
Teams contribute incrementally; everyone benefits immediately.

🧩 Incremental growth

Anyone adds a new skill or agent → next `./setup.sh --update` and it's available to everyone. No deployment, no coordination.

🤖 Orchestrator-driven

The orchestrator sees all registered agents and skills, and delegates automatically — *including ones added after it was written.*

📚 Skills as shared knowledge

Reusable reference docs (SAIL patterns, a11y rules, Aurora specs) any agent can load on demand. Write once, used by all.

🔄 Composable agents

Sub-agents invoke other sub-agents. developer can call sql-forge; qe-agent can call a11y-fixer. Modular composition.

The compound effect: every new skill or agent makes every *other* agent smarter. A new "performance-patterns" skill instantly improves developer, code-reviewer, and qe-agent — without touching their code.

ACT 4 · WHY THIS DESIGN WINS

ADVANTAGE 2 — PORTABILITY

Platform agnostic — write once, configure anywhere

The entire knowledge layer is platform-independent. Only the thin setup script is platform-specific.

Why this matters: AI tooling moves fast. If a better platform emerges next quarter, we don't rewrite Solutions OS — we write a ~100-line setup script. All knowledge and workflows carry forward. **Zero vendor lock-in.**

ACT 4 · WHY THIS DESIGN WINS

SUPPORTING INFRASTRUCTURE

Centralized environment registry

Teams work across many Appian environments. Today credentials are scattered across individual configs. The registry centralizes URLs (safe to commit) and keeps secrets separate.

Shared registry (in repo)

```
// environments.json
{
  "gam-dev2": {
    "url": "https://…dev2.appianpreview.com",
    "products": ["source-selection", "vendor-mgmt"]
  },
  "cms-dev": {
    "products": ["case-management-studio"]
  }
}
```

How agents use it

Auto-select working on CMS? picks cms-dev automatically

Multi-env "compare schema dev vs staging" → queries both

Secure URLs in registry; API keys in private creds file

Switching "deploy to staging" → resolves env by name

No more hardcoded URLs in power configs. No more "which environment am I pointed at?"

ACT 4 · WHY THIS DESIGN WINS

KNOWLEDGE ARCHITECTURE

Two knowledge layers, one platform

Solutions OS serves two distinct kinds of intelligence — they answer fundamentally different questions.

📚 Product Knowledge

"What should we build and why?"

Source: `products/` folders (T.I.M.E.)

Contains: vision, personas, specs, decisions

Indexed by: Kiro KB engine (auto-updates on git pull)

Used by: orchestrator, product-owner

🔧 Application Knowledge

"What exists in the code and how does it work?"

Source: Solutions Parser → GitLab KB (JSON)

Contains: objects, dependencies, schema, versions

Accessed via: Intelligence Server tools (on demand)

Used by: developer, sql-forge, code-reviewer

One **Solutions Parser** feeds both: Appian .zip packages → JSON for the Cloud Plane, and live KB data for the Live Plane.

Product knowledge auto-updates on git pull; application knowledge refreshes via the CI pipeline.

ACT 5 · EXECUTION

PROVEN FEASIBILITY

All 20 SWAT-a-Palooza projects fit

This isn't theoretical. Every hackathon project maps cleanly into the revamped architecture — no project needs a rewrite, just repositioning.

A11Y 2

A11Y Fixer → **a11y-fixer agent**
A11y Audit → **shared skill**

DATA 2

SQL Forge + DataForge → **sql-forge agent**

UX 4

UX Enhancements, Kiro→FigJam, Spec→Slides, SAIL Canvas → **ux-designer**

DEV 5

Sweep, Perf-Profiler, a!migo, SAIL→SQL, Assert → **developer**

QE 2

Test Execution Agent → **qe-agent**
Expression Assert → **developer**

DOCS 5

Feature Doc Genie, ERD Gen, T.I.M.E., KB Maintenance, Sprint Report

Why every project fits: unified read tools give all agents both data planes · the orchestrator auto-routes · the environment registry handles multi-env targeting · skills share reference knowledge across agents.